

I will apply the **API composition** software pattern to the **User Dashboard** feature.

Name	Andrew ID	Team ID
Tyler Ma	haokaim	Team 02

1. Feature Context and Pattern Selection

The dashboard in this project is more than a simple home page. It is the user’s main control center for logging in with a third-party provider, recharging credits through a third-party payment API, viewing which learning content has been unlocked, and checking learning history and current progress. Even though the overall platform also includes capabilities such as Read Aloud, Image Generation, Comprehension Questions, and Word Phonics & Pronunciation, the dashboard is not responsible for implementing those feature logics. Its responsibility is to provide a coherent view of the user’s state across those modules.

I selected the API Composition pattern because the central design problem of the dashboard is data aggregation across multiple bounded contexts or services. A dashboard request may need authentication data, wallet or credit data, payment transaction data, content entitlement data, and progress summaries. In a realistic team project or microservices-oriented architecture, these concerns should not be collapsed into one large service or one shared database table. Instead, the dashboard should depend on a dedicated composition layer that queries the required services and then returns one dashboard-specific response model.

This pattern is also a better fit than a purely object-level Gang of Four pattern for this feature. A GoF pattern such as Strategy or Facade could help with a smaller part of the implementation, but the main architectural challenge here is coordinating distributed data sources to support one user-facing view. API Composition addresses that problem directly and therefore represents a meaningful and non-artificial pattern choice.

2. Why the Pattern Is Suitable

API Composition is suitable for this feature for four reasons. First, it matches the dashboard’s true responsibility: aggregation rather than ownership of every underlying business capability. Second, it preserves separation of concerns because payment, wallet, progress tracking, and content entitlement remain owned by their corresponding services. Third, it improves the user experience because the web client can request one dashboard payload instead of coordinating many service calls itself. Fourth, it supports future growth. If the platform later adds badges, parent-facing analytics, subscription tiers, or voice settings for narrated stories, the dashboard can incorporate additional summaries without changing the basic interaction model between frontend and services.

The pattern also aligns well with the team-project setting described in the assignment. Each teammate may own a different feature or service. API Composition lets the dashboard integrate those features without absorbing their internal logic. In other words, the dashboard remains a consumer of feature summaries rather than becoming the implementation home for every feature in the project.

3. Benefits and Potential Drawbacks

Benefit	Why it matters for the dashboard
Single tailored endpoint	The web client receives one dashboard response instead of calling

	several internal services directly.
Loose coupling	Each downstream service keeps its own data model and implementation details; the dashboard only depends on stable client interfaces.
Clear ownership boundaries	The dashboard team owns aggregation logic, while other teams can continue owning progress, payment, or entitlement logic.
Easier evolution	New dashboard widgets can be added by extending the composer and the response model instead of redesigning the entire frontend integration.
Centralized resilience	Timeouts, fallback values, caching, and partial-response handling can be implemented once in the composition layer.

The pattern also introduces trade-offs that should be acknowledged.

Potential drawback	Impact and mitigation
Higher latency risk	Because the composer must call multiple services, response time can increase. This can be reduced through parallel requests, caching, and selective loading of noncritical widgets.
Partial failures	One downstream service may fail while others succeed. The dashboard should therefore support fallback values such as “progress temporarily unavailable” instead of failing the entire page.
More orchestration code	The composition layer adds implementation complexity. However, that complexity is justified because it prevents even greater complexity in the frontend and across service boundaries.
Risk of an oversized composer	If every cross-cutting concern is pushed into the dashboard composer, it can become a god service. The mitigation is to keep it focused on view composition only and not let it absorb core business rules from other services.

4. Class Diagram Design and Annotation

